Verified compiler for a language with control effects

(Weryfikacja kompilatora dla języka programowania z efektami sterowania)

Paweł Dybiec

Praca magisterska

Promotor: Piotr Polesiuk

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

12 marca 2026

Abstract

We present a type system for a language with labeled delimited control operators. It utilizes polymorphic types and effects to allow expressions parametrized by effects. Our goal is to formally prove the correctness of such a language. Formalisation work was done in Agda programming language.

Tematem tej pracy jest stworzenie i zweryfikowanie języka z operatorami sterowania dla kontynuacji ograniczonych. Język ten używa polimorficznych typów aby pozwolić na wyrażenie obliczeń generycznych wobec pewnych efektów. Celem pracy jest zweryfikowanie poprawności definicji takiego języka. Formalizacja została przeprowadzona w języku programowania Agda.

Contents

1 Introduction **7**

2 Surface language **9**

2.1 Syntax 9

2.2 Typing judgements 10

3 Runtime language **13**

3.1 Runtime language 13

3.2 Runtime embedding 15

4 Semantics **17**

4.1 Values 17

4.2 Frames 17

4.3 Reduction 19

4.4 Progress 21

5 Conclusion **23**

Bibliography **25**

Chapter 1

Introduction

Control operators have been present in languages for a long time. One notable example is the SCHEME programming language which originated in the 70s[1] and provides a *call/cc* operator. Such operators enable non-local jumps, which might be familiar to users of imperative languages with the goto operator. But those operators unlike goto are handled by capturing continuation, which is stored as first-class value, able to be passed, called or dropped. Operator *call/cc* when called, catches the whole rest of the program in the continuation. When this continuation is called, it aborts the rest of the program and replaces it with this continuation. Thus in the example program `10 + (call/cc (λk . k (2 + (k 1))))` only inner *k* would be called and the result of the whole program would be 11.

Operators for delimited control introduced by Danvy and Filinski[2] and independently by Felleisen[3] allow more precise control of continuations. They usually come in pairs: for example, the operator `reset` delimits continuations caught by `shift`. This allows localized use of control effects, as the scope of the captured continuation is much more visible, while still allowing non-local execution, which enables interesting computational effects such as emulating non-determinism, exceptions and failure. Usually such calculi jump from `shift` to the closest enclosing *reset*. Reduction rule looks like this: `(reset E[(shift k expr)])` ⇒ `(reset ((λk.expr) (λv.(reset E[v]))))` therefore, example `(reset (+ 1 (shift k (k (k 0)))))` would call *k* twice and result in value 2.

Danvy and Filinski[4] also describe similar operators $\texttt{shift}_0$ and $\texttt{reset}_0$. Their reduction does not introduce outermost $\texttt{reset}_0$, therefore subsequent calls to $\texttt{shift}_0$ remove innermost $\texttt{reset}_0$ one by one. Their example showcases this behaviour $\texttt{reset}_0\texttt{(3 + reset}_0\texttt{(4 * shift}_0\texttt{ k in shift}_0\texttt{ c in E))}$. The continuation *k* would be bound to `λx.4*x` and *c* to `λx.3+x`. In addition Dybvig et al.[5] show the formalisation of delimited control with multiple prompts where labels are used to match the corresponding operators.

The aim of this thesis is to formalise a typed version of such a calculus with polymorphic types (similar to Piróg, Polesiuk and Sieczkowski[6]), but for labeled delimited continuations (like Ikemori, Cong and Masuhara[7]).
Compared to Ikemori et al., the main difference in our work is the use of first-class labels for delimited continuations, similarly to how Xie et al.[8] used them for algebraic effects. Balik and Polesiuk presented such a language[9] which currently serves as the core of Fram programming language. Formalising such a calculus could serve as a step in to demonstrate soundness of Fram's implementation.

The soundness of the presented language is proved using the standard technique of progress and preservation[10][11]. Formalisation work was done in Agda 2.8[12] using `agda-stdlib`[13], without use of third-party dependencies. The accompanying code archive, available at https://github.com/dyniec/mgr, also features nix derivation describing the exact environment used to build both package and this document, to ensure reproducibility.

Chapter 2

Surface language

2.1 Syntax

We start with presenting the syntax of the surface language. We use kinds to distinguish between types and effects.

```
data Kind : Set where
  T : Kind
  E : Kind
```

Types and effects are defined as mutually recursive. A type is either a type variable `ttv`, an arrow, a `forallt` or a label `L eff at A / E`. We represent variables and type variables with de Bruijn indices. Effects datatype stores a list of effects (represented as types), it's used in arrow and forall constructors of type to mark effects of computation underneath. It will be also used in typing judgements to mark effects available in computation. Constructor `L` represents type of label expressions. It just stores an effect `eff` represented by a label and type and effect of delimited computation labelled by this label.

```
module Types where
  Id : Set
  Id = ℕ
  data Type : Set
  Effects : Set
  data Type where
    ttv : Id → Type
    _-_>_ : Type → Effects → Type → Type
    forallt : Kind → Type → Effects → Type
    L_at_/_ : Type → Type → Effects → Type
  Effects = List Type
open Types
```

Constructors of expressions such as `var`, `lam`, `app`, `tlam`, and `tapp` behave as usual. `var` is a de Bruijn indexed variable, `lam` is a lambda abstraction, `app` is a function application, `tlam` is a type abstraction, storing a kind of abstracted type and `tapp` is type application. Other constructors are responsible for continuations and labels. Constructor `new` binds labels used by $\texttt{shift}_0$ and $\texttt{reset}_0$ expressions to pair them up.

Constructor `shift₀` stores the label and expression (parametrized by continuation) that replaces whole delimited computation. The last constructor of expressions is `reset₀ e (v . en) e'` which has 3 arguments, the last one `e'` is label that is used to match up `reset₀`s with `shift₀`s. First argument `e` is delimited computation where `shift₀` can be used. So when `shift₀ k.es` is being evaluated, it aborts enclosing up to corresponding `reset₀`, replacing it with `es` is being evaluated with `k` bound to continuation representing evaluation context between `shift₀` and its `reset₀`. Second argument `x. en` is used when no corresponding `shift₀` aborts during evaluation of `e`. If `e` evaluates to `v`, then whole `reset₀` reduces to `en` where `x` is bound to `v`.

```
module Expr where
  data Expr : Set where
    var : ℕ → Expr
    lam : Expr → Expr
    app : Expr → Expr → Expr
    tlam : Kind → Expr → Expr
    tapp : Expr → Type → Expr
    new : Expr → Expr
    shift₀ : Expr → Expr → Expr
    reset₀ : Expr → Expr → Expr → Expr
open Expr
```

2.2 Typing judgements

Typing contexts are represented as lists of types, and contexts for type variables are represented as lists of kinds. Type or kind judgements for membership have Peano numbers structure.

```
module Typing where
  infixl 5 _,_
  data Context : Set where
    ∅ : Context
    _,_ : Context → Type → Context

  data TContext : Set where
    ∅ : TContext
    _,_ : TContext → Kind → TContext
```

Expressions are extrinsically typed, thus typing judgements are represented separately. Typing rules for `var`, `lam`, `app`, `tlam`, and `tapp` are defined mostly as usual. The noticeable difference is that value expressions are generic over effects they execute.

```
data _,_⊢_∶_/_ : TContext → Context → Expr → Type → Effects → Set where
  ⊢var : ∀ {Γ Δ x A E}
    → Γ ∋ x ∶ A
    ----------------------
    → Δ , Γ ⊢ var x ∶ A / E

  ⊢lam : ∀ {Γ Δ e A B E F}
    → Δ , (Γ , A) ⊢ e ∶ B / E
    ---------------------------------
    → Δ , Γ ⊢ lam e ∶ A - E > B / F
```

Weakening utilises subtyping and subeffecting relations. Effect is defined as subeffect of another effect, if its list representation is a subsequence of the other

effect. This does not allow for reordering or merging occurrences of same entries. Subtyping is defined recursively using subeffecting. That is, subtyping for arrows uses subeffecting on its effects, and uses subtyping recursively on input and output type of arrow while keeping proper variance. Subtyping for foralls is defined in a straightforward manner. Subtyping for labels is not implemented, as they precisely bind type and effects of delimiter.

```
⊢weak : ∀ {Γ Δ e A A' E E'}
  → Δ ⊢ A <t: A'
  → Δ ⊢ E <: E'
  → Δ , Γ ⊢ e : A / E
  ---------------------
  → Δ , Γ ⊢ e : A' / E'

⊢app : ∀ {Γ Δ e1 e2 A B E}
  → Δ , Γ ⊢ e1 : A - E > B / E
  → Δ , Γ ⊢ e2 : A / E
  ----------------------------
  → Δ , Γ ⊢ app e1 e2 : B / E

⊢forall : ∀ {Γ Δ e k A E F}
  → (Δ , k) , Γ ⊢ e : TypeSubst.bump A / TypeSubst.bump' E
  ------------------------------------
  → Δ , Γ ⊢ tlam k e : forallt k A E / F

⊢tapp : ∀ {Γ Δ e k A T E}
  → Δ ⊢ T :te k
  → Δ , Γ ⊢ e : forallt k A E / E
  ------------------------------------------------
  → Δ , Γ ⊢ tapp e T : A TypeSubst.[ T ] / (E TypeSubst.effs[t T ])
```

The `new` construct introduces a new type variable and a variable that represent respectively an effect and a label. The label type stores the effect bound by `new` (here `ttv zero`) and `A1 / E2` representing a type and an effect of delimited computation.

```
⊢new : ∀ {Γ Δ e A A1 E E1}
  → Δ ⊢ A1 :t
  → Δ ⊢ E1 :effs
  → (Δ , Kind.E) , (Γ , (L ttv zero at A1 / E1)) ⊢ e : TypeSubst.bump A / TypeSubst.bump' E
  ----------------------
  → Δ , Γ ⊢ new e : A / E
```

Constructor `shift₀ e' (k . e)` uses only one effect `E` represented by label `e'`. For it to be a properly typed expression inside shift, the constructor binds an extra variable `k`, which is where continuation will be plugged. Therefore the type of this expression `e` should have the same type and effect as stored in the label type. Its typing context should be expanded to account for continuation, which type should be an arrow from `shift₀` type to type and effect of whole delimited computation. Since during evaluation `reset₀` will be removed, effect `E` it has introduced will not be present in direct subexpressions of `shift₀`.

```
⊢shift₀ : ∀ {Γ Δ e e' A A' E E'}
  → Δ ⊢ E :e
  → Δ , Γ ⊢ e' : (L E at A' / E') / nil
  → Δ , (Γ , A - E' > A' ) ⊢ e : A' / E'
  ---------------------------------------
  → Δ , Γ ⊢ shift₀ e' e : A / (E :: nil)
```

The `reset₀ e (v. en) e'` constructor has three parameters. The first one is the expression `e` that will have access to the effect, so its list of effects is expanded.

The second one is continuation `v . en` that will handle the value `v` returned from the first argument `e`. And third is the label `e'`, whose effect is the same one as one introduced in `e`. Label type also stores the type and effects of the whole delimited computation.

```
⊢reset₀ : ∀ {Γ Δ e e' en A A' E E'}
  → Δ ⊢ E ꞉e
  → Δ , Γ ⊢ e ꞉ A / (E :: E')
  → Δ , (Γ , A) ⊢ en ꞉ A' / E'
  → Δ , Γ ⊢ e' ꞉ (L E at A' / E') / nil
  ------------------------------------
  → Δ , Γ ⊢ reset₀ e en e' ꞉ A' / E'
```

Now that the surface language and its typing rules have been defined for it, the next chapter will consider how it could be evaluated.

Chapter 3

Runtime language

3.1 Runtime language

Grammar defined in Chapter 2 does not produce any expression other than variable of label type, while `shift₀` and `reset₀` require the same labels. Therefore we see two options to define a reduction relation: going under `new` binders or introducing label expressions. We decided to go with the second option and define another runtime expression language expanded by the notion of label values.

When the `new` expression is evaluated, all occurrences of variables bound by it will have been replaced with the newly allocated label value. That means reduction relation would need to be defined using an allocator state.

To ensure type safety, we will expand type syntax to include allocated effects and add new effect context to typing judgments, which maps allocations to type and effect of the delimiter.

```
module Types_ where
  Id : Set
  Id = ℕ
  Label = ℕ
  data Type : Set
  Effects : Set
  data Type where
    ttv : Id → Type
    _-_>_ : Type → Effects → Type → Type
    forallt : Kind → Type → Effects → Type
    L_at_/_ : Type → Type → Effects → Type
    Effect : Label → Type
  Effects = List Type
open Types_
```

Most of the constructors in `RExpr` are the same as in `Expr`. Labels runtime values are represented by natural numbers.

```
module RuntimeExpr where
  data RExpr : Set where --runtime version
    var : ℕ → RExpr
    lam : RExpr → RExpr
    app : RExpr → RExpr → RExpr
    tlam : Kind → RExpr → RExpr
    tapp : RExpr → Type → RExpr
    new : RExpr → RExpr
```

```
    shift₀ : RExpr → RExpr → RExpr
    reset₀ : RExpr → RExpr → RExpr → RExpr
```

A separate term for the label stores the allocated label identifier.

```
label : Label → RExpr
```

Since the grammar of types has changed, we need to redefine contexts. This means almost all typing judgements need to be redefined. A new context is introduced that maps label values to type and effects of the delimiter. This technique is reminiscent of technique described by Pierce[14, Chapter 13], typing rules take maps from locations to types in order to ensure allocations' type safety. This context is represented by a finite vector of types and effects. As they are only used during evaluation, the types and effects have no free variables.

```
module Typing where
  open RuntimeExpr
  infixl 5 _,_
  data Context : Set where
    ∅ : Context
    _,_ : Context → Type → Context


  data TContext : Set where
    ∅ : TContext
    _,_ : TContext → Kind → TContext


  push : Kind → TContext → TContext
  push k Δ = Δ , k
  EContext = Σ[ n ∈ ℕ ] Data.Vec.Vec (Type × Effects) n
  pattern ∅' = 0 ,, Data.Vec.[]
```

Most of typing judgements work as before. We omit the discussion of the ones that show up in surface language, as they simply pass extra context for effects throughout the rules.

```
private
  variable
    Γ : Context
    Δ : TContext
    Θ : EContext
    e e1 e2 : RExpr
    A B : Type
    E E' F : Effects
    k : Kind
infix 4 _;_;_⊢_∶_/_
data _;_;_⊢_∶_/_ : TContext → EContext → Context → RExpr → Type → Effects → Set where
```

Since the labels are represented by natural numbers, ensuring correct label type relies on a simple lookup in an appropriate context.

```
⊢label : ∀ {n }
  → Θ ∋l n ∶ A / E
  ------------------------------------------
  → Δ ; Θ ; Γ ⊢ label n ∶ (L (Effect n) at A / E) / F
```

3.2 Runtime embedding

We defined embedding of the surface language in the runtime language one, and showed the proof that such embedding preserves typing judgements. Most of the proof is not displayed here, as it is repetitive, that is, it goes through almost all judgement types. The type of main proof is presented below.

```
runtime-types : ∀ {Δ Γ e T E}
  → Δ Types.Typing., Γ ⊢ e ꞉ T / E → (runtimeΔ Δ ; ∅' ; runtimeΓ Γ ⊢ (runtime e) ꞉ rt-t T / rt-t' E)
```

Chapter 4

Semantics

This chapter defines the reduction relation and shows its soundness.

4.1 Values

Only abstractions, type abstractions and labels are considered values. Since the values themselves do not perform any effects, they have a `nil` effect. But rules for all of them have a built-in weakening. We can use that to generalize their type and perform a substitution where any effect is expected.

```
data Value : RExpr → Set where
  vlam : ∀ { e } → Value (lam e)
  vLam : ∀ { k e } → Value (tlam k e)
  vlab : ∀ { n } → Value (label n)
gvalue : ∀ {Δ Θ Γ T E e} → (Value e) → (Δ ; Θ ; Γ ⊢ e ꞉ T / E) → ∀ {F} → (Δ ; Θ ; Γ ⊢ e ꞉ T / F)
gvalue vlam (⊢lam t) = ⊢lam t
gvalue vLam (⊢forall t) = ⊢forall t
gvalue vlab (⊢label x) = ⊢label x
gvalue {Δ} v (⊢weak x x₁ x₂) {F} = (⊢weak x ( <꞉e-refl {Δ} {F}) (gvalue v x₂))
```

4.2 Frames

In this thesis, the term "frame" stands for what is usually an evaluation context in literature. This term was selected to be sufficiently different from typing context. The frame represents parts between $\texttt{reset}_0$s, or between $\texttt{reset}_0$ and $\texttt{shift}_0$. Frame type is parametrized by Θ Γ, that is, the typing context outside of frame, a T type of the hole. It's also indexed by the whole frame type and effects, and the effects of the hole. Frames are intrinsically typed, thus they also store type judgements of subexpressions. They are defined in such a way to reduce repetition. Otherwise we would need to introduce typing judgements for frames, and then prove type preservation for every operation such as plugging or composition.

```
data Frame (Θ : EContext) (Γ : Context) (T : Type) : Effects → Type → Effects → Set where
  fempty : ∀ {Eff}
  ----------------------------
```

```
    → Frame Θ Γ T Eff T Eff
  fapp₁ : ∀ {A B Eff E} → Frame Θ Γ T Eff (A - Eff > B) E
    → (e : RExpr) → { ∅ ; Θ ; Γ ⊢ e ⦂ A / Eff }
    -------------------------
    → Frame Θ Γ T Eff B E
  fapp₂ : ∀ {A B Eff E} → (e : RExpr) → { v : Value e}
    → { ∅ ; Θ ; Γ ⊢ e ⦂ ( A - Eff > B) / Eff }
    → Frame Θ Γ T Eff A E
    ---------------------------------------------------------
    → Frame Θ Γ T Eff B E
  ftapp : ∀ {A B Eff E k}
    → ∅ ; Θ ⊢ B ⦂te k
    ---------------------------------------------------------
    → Frame Θ Γ T Eff (forallt k A Eff) E
    → Frame Θ Γ T (Eff TypeSubst.effs[t B ]) (A TypeSubst.[ B ] ) E
  freset-label : ∀ {A E A' Eff C}
    → (e en : RExpr)
    → ∅ ; Θ ⊢ C ⦂e
    → ∅ ; Θ ; Γ  ⊢ e ⦂ A' / (C :: Eff)
    → ∅ ; Θ ; (Γ , A')  ⊢ en ⦂ A / Eff
    → Frame Θ Γ T nil (L C at A / Eff) E
    -------------------------
    → Frame Θ Γ T Eff A E
  fshift-label : ∀ {A E A' E' C}
    → (e : RExpr)
    → ∅ ; Θ ⊢ C ⦂e
    → ∅ ; Θ ; (Γ , A - E' > A' ) ⊢ e ⦂ A' / E'
    → Frame Θ Γ T nil (L C at A' / E') E
    ----------------------------------
    → Frame Θ Γ T (C :: nil) A E
```

Definition and types for frame plugging and composition:

```
plug : ∀ {Θ Γ T Eff A E}
  → Frame Θ Γ T Eff A E
  → (e : RExpr) → ∅ ; Θ ; Γ ⊢ e ⦂ T / E
  → Σ[ res ∈ RExpr ] (∅ ; Θ ; Γ ⊢ res ⦂ A / Eff)
_∘f_ : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → Frame Θ Γ B Eff A Eff'
  → Frame Θ Γ C Eff' B Eff''
  → Frame Θ Γ C Eff A Eff''
```

We prove how plugging and composition relate. Plugging an expression into one frame and result of that into another frame results in the same value and type as plugging the very same expression into a composition of two frames.

```
∘f-lemma : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → (f1 : Frame Θ Γ B Eff A Eff' )
  → (f2 : Frame Θ Γ C Eff' B Eff'' )
  → (e : RExpr) → (t : ∅ ; Θ ; Γ ⊢ e ⦂ C / Eff'')
  → plug ( f1 ∘f f2) e t
  ≡ ((λ x → plug f1 (Data.Product.proj₁ x) (Data.Product.proj₂ x))(plug f2 e t))
```

A metaframe stores the whole evaluation context, split into frames separated by resets. Type parameters and indices work in the same way as in the frame. Unlike the frame, however the metaframe now stores `reset₀`s, so the lists of effects inside and outside the frame may differ. This means that their difference represents a list of effects handled by the frame.

We will prove that for well-typed expressions they either are redex, value or they decompose into metaframe and `shift₀` expression. This and observation about diff-lists, will allow us to prove that if pure well-typed expression decomposes into shift and metaframe, then this metaframe should handle `shift₀`'s effect. Thus it has matching `reset₀` inside of frame, therefore whole expression is also a redex.

```
data Metaframe (Θ : EContext) (Γ : Context) (T : Type) (Eff : Effects)
  : Type → Effects → Set where
  mfempty : Metaframe Θ Γ T Eff T Eff
  mfreset : ∀ { A B C Eff' }
    → (l : Label)
    → ∅ ; Θ ⊢ C ꞉e
    → ∅ ; Θ ; Γ ⊢ label l ꞉ (L C at B / Eff) / nil
    → (e : RExpr) → (∅ ; Θ ; Γ , A ⊢ e ꞉ B / Eff)
    → Metaframe Θ Γ T (C :: Eff) A Eff'
    ---------------------------------
    → Metaframe Θ Γ T Eff B Eff'
  mframe : ∀ {A Eff' B Eff'' }
    → Frame Θ     Γ A Eff B Eff'
    → Metaframe Θ Γ T Eff' A Eff''
    ----------------------------------------
    → Metaframe Θ Γ T Eff B Eff''
```

Similarly to frames, metaframes can be lifted and plugged into arbitrary contexts. They can also be composed with simple frames.

```
↑m : forall {Θ A B Eff Eff' Γ' Γ}
  → Metaframe Θ Γ       A Eff B Eff'
  → Metaframe Θ (Γ' ⧺ Γ) A Eff B Eff'
mplug : ∀ {Θ Γ T Eff A E}
  → Metaframe Θ Γ T Eff A E
  → (e : RExpr) → ∅ ; Θ ; Γ ⊢ e ꞉ T / E
  → Σ[ res ∈ RExpr ] (∅ ; Θ ; Γ ⊢ res ꞉ A / Eff)
_∘m_ : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → Metaframe Θ Γ B Eff A Eff'
  → Metaframe Θ Γ C Eff' B Eff''
  → Metaframe Θ Γ C Eff A Eff''
_f∘m_ : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → Frame Θ Γ B Eff A Eff'
  → Metaframe Θ Γ C Eff' B Eff''
  → Metaframe Θ Γ C Eff A Eff''
```

4.3 Reduction

Since labels need to be allocated, the reduction relation is defined in terms of the expression and state. The state itself is just the next label to be allocated. As metaframes are intrinsically typed, we need to provide judgements representing expressions well-typedness.

We define reduction relation parametrised by both expressions and typing judgemenets before and after reduction. With this approach, the definition itself is a proof of type preservation.

```
data _;_↦_;_;_ : (e : RExpr)
  → (∅ ; Θ ; ∅ ⊢ e ꞉ A / E)
  → (Θ' : EContext)
  → (e' : RExpr)
  → (∅ ; Θ' ; ∅ ⊢ e' ꞉ A / E) → Set where
```

Reduction of `new` constructor replaces bound variable with allocated label, and type variable with entry in store typing. It also updates the effects context, as we just allocated an effect.

```
↦new : ∀ {e Θ  E T A1 E1}
  → (te : (∅ , Kind.E) ; Θ ;(∅ , L ttv zero at A1 / E1) ⊢ e ꞉ TypeSubst.bump T / TypeSubst.bump' E)
  → {tn : ∅ ; Θ ; ∅ ⊢ new e ꞉ T / E}
  → new e ; tn ↦ pb Θ (A1 ,′ E1) ; (new-subst te tn) .proj₁ ; new-subst te tn .proj₂
```

Reduction of application applied to abstraction, and type application to type abstraction are defined in terms of substitution.

```
lam-app : ∀ {e V Θ A B E }
  → {te : ∅ ; Θ ; (∅ , A) ⊢ e ꞉ B / E}
  → {tv : ∅ ; Θ ; ∅ ⊢ V ꞉ A / E}
  → Value (lam e)
  → (v : Value V)
  → app (lam e) V ; ⊢app (⊢lam te) tv ↦ Θ ;
  (RExprSubstTyped._[_] e V {te = te} {te1 = (gvalue v tv)}) .proj₁ ;
  (RExprSubstTyped._[_] e V {te = te} {te1 = (gvalue v tv)}) .proj₂

tlam-tapp : ∀ {k e T }
  → Value (tlam k e)
    → (tt : ∅ ; Θ ⊢ T ꞉te k)
    → (tv : ∅ ; Θ ; ∅ ⊢ (tlam k e) ꞉ forallt k A E / E)
  → tapp (tlam k e) T ; ⊢tapp tt tv ↦ Θ ; e RExprSubst.e[t T ] ; tapp-subst tt tv
```

Reduction of reset where inner computation returns value, just substitutes returned value in success continuation.

```
reset₀-vl : ∀ {V e' en Θ A B E T }
  → ( v : Value V)
  → {tt : ∅ ; Θ ⊢ T ꞉e }
  → {tv : ∅ ; Θ ; ∅ ⊢ V ꞉ A / T :: E}
  → {ten : ∅ ; Θ ; (∅ , A) ⊢ en ꞉ B / E}
  → {tl : ∅ ; Θ ; ∅ ⊢ e' ꞉ (L T at B / E) / nil }
  → reset₀ V en e' ; (⊢reset₀ tt tl tv ten)
  ↦ Θ ; RExprSubstTyped._[_] en V
  {te = ten} {te1 = (gvalue v tv)} .proj₁ ;
  RExprSubstTyped._[_] en V {te = ten} {te1 = (gvalue v tv)} .proj₂
```

Reduction `shift` and `reset` is the most complicated reduction rule. We use metaframes and equivalence relation to express decomposition of expression into a `shift` and a continuation. We replace whole computation up to and including reset with computation under `shift` of which first argument is replaced with captured continuation from `shift` up to and including `reset`.

```
reset₀-k : ∀ {Θ es e' El A B C E' e en}
  → {elabel : ∅ ; Θ ⊢ El ꞉e }
  → {tlabel : ∅ ; Θ ; ∅ ⊢ e' ꞉ (L El at B / E') / nil }
  → {tes : ∅ ; Θ ; (∅ , A - E' > B ) ⊢ es ꞉ B / E' }
  → {tshift : ∅ ; Θ ; ∅ ⊢ shift₀ e' es ꞉ A / (El :: nil)}
  → {f : Metaframe Θ ∅ A (El :: E') C (El :: nil) }
  → e ≡ mplug f (shift₀ e' es) tshift .proj₁
  → {te : ∅ ; Θ ; ∅ ⊢ e ꞉ C / (El :: E') }
  → {ten : ∅ ; Θ ; (∅ , C) ⊢ en ꞉ B / ( E') }
  → reset₀ e en e' ; ⊢reset₀ elabel tlabel te ten
  ↦ Θ ; (RExprSubstTyped._[_] es
  (lam (reset₀ ((mplug (↑m {Γ' = ∅ , A} f) (var 0 ) (⊢var Z)) .proj₁) en e'))
  {te = tes} {te1 = gvalue {E = E'} vlam (⊢lam (⊢reset₀ elabel (e↑ tlabel)
    (e↑ (mplug (↑m f) (var 0) (⊢var Z) .proj₂)) (e↑ ten)))} .proj₁)
  ; (RExprSubstTyped._[_] es
  (lam (reset₀ ((mplug (↑m {Γ' = ∅ , A} f) (var 0 ) (⊢var Z)) .proj₁) en e'))
  {te = tes} {te1 = gvalue {E = E'} vlam (⊢lam (⊢reset₀ elabel (e↑ tlabel)
    (e↑ (mplug (↑m f) (var 0) (⊢var Z) .proj₂)) (e↑ ten)))} .proj₂)
```

Since the simple reduction above is defined directly on redexes, we introduce → that represents the reduction within the metaframe.

```
data _;_→_;_;_ : (e : RExpr)
  → (∅ ; Θ ; ∅ ⊢ e ꞉ A / E)
  → (Θ' : EContext)
  →(e' : RExpr)
  → (∅ ; Θ' ; ∅ ⊢ e' ꞉ A / E) → Set where
  →frame : ∀ { Θ' e e' e1 e1' A T Eff } → (f : Metaframe Θ ∅ A Eff T E)
    → (t1 : ∅ ; Θ ; ∅ ⊢ e1 ꞉ A / E )
    → (t1' : ∅ ; Θ' ; ∅ ⊢ e1' ꞉ A / E )
```

```
  → e1 ; t1 ↦ θ' ; e1' ; t1'
  → (θstep : ∀ {A E B Eff} → Metaframe θ ∅ A E B Eff → Metaframe θ' ∅ A E B Eff )
  → (mplug f e1 t1) .proj₁ ≡ e
  → (te : ∅ ; θ ; ∅ ⊢ e ꞉ A / E)
  → (mplug (θstep f) e1' t1') .proj₁ ≡ e'
  → (te' : ∅ ; θ' ; ∅ ⊢ e' ꞉ A / E)
  → e ; te → θ' ; e' ; te'
```

4.4 Progress

Since the subject reduction is builtin into the definition of the reduction, we only need to prove progress to ensure type safety. We introduce progress datatype to represent progress, well typed expression is either value, or can reduce.

```
data Progress : (e : RExpr) → (∅ ; θ ; ∅ ⊢ e ꞉ A / nil) → Set where
  done : ∀ {e} → Value e
    → (te : ∅ ; θ ; ∅ ⊢ e ꞉ A / nil)
    → Progress e te
  step : ∀ {e1 e2 θ'}
    → (te1 : ∅ ; θ ; ∅ ⊢ e1 ꞉ A / nil)
    → (te2 : ∅ ; θ' ; ∅ ⊢ e2 ꞉ A / nil)
    → e1 ; te1 → θ' ; e2 ; te2
    → Progress e1 te1
```

Decompose datatype is similar to progress.Since we are building it upwards we might encounter a `shift₀`, but not yet its corresponding reset. Because of that we introduce an extra constructor that represents `shift₀` and its surrounding frame.

```
data Decompose : (e : RExpr) → (∅ ; θ ; ∅ ⊢ e ꞉ A / E) → Set where
  de-simpl-redex : ∀ {e1 e2 θ'}
    → (te1 : ∅ ; θ ; ∅ ⊢ e1 ꞉ A / E)
    → (te2 : ∅ ; θ' ; ∅ ⊢ e2 ꞉ A / E)
    → e1 ; te1 → θ' ; e2 ; te2
    → Decompose e1 te1
  de-val : ∀ {e} → Value e
    → (te : ∅ ; θ ; ∅ ⊢ e ꞉ A / E)
    → Decompose e te
  de-shift : ∀ { T Eff A Eff' es es' e l t}
    → (f : Metaframe θ ∅ T Eff A Eff' )
    → shift₀ (label l) es' ≡ es
    → Data.Product.proj₁ (mplug f es t) ≡ e
    → (te : ∅ ; θ ; ∅ ⊢ e ꞉ A / E)
    → (ts : ∅ ; θ ; ∅ ⊢ es ꞉ T / E')
    → Decompose e te
```

Proof of progress has a type of `progress : ∀ {A Δ Effs} → (s : State) → (e : RExpr) → (t : Δ ; ∅ ⊢ e ꞉ A / nil) → Progress s e`. In such proof we use auxiliary struct `Decompose` of which builder `decompose` walks down well typed expression recursively until it has reached either value, simple reduction (app, tapp, new), or shift, and its surrounding metaframe.

In case of shift, such a metaframe by construction should have an effect handler that has the same effect as shift. So we can construct `rest₀-k` and surrounding metaframe. Other cases would either be immediate value, or simple reduction in context.

```
decompose : ∀ {A Effs} → (e : RExpr) → (t : ∅ ; θ ; ∅ ⊢ e ꞉ A / Effs) → Decompose e t
progress : ∀ {A } → (e : RExpr) → (t : ∅ ; θ ; ∅ ⊢ e ꞉ A / nil) → Progress e t
```

Chapter 5

Conclusion

This thesis presents a language with labeled delimited continuations together with a partial language formalisation. Two term languages are defined, alongside typing judgements for them and translations between them. Typing is preserved when embedding surface language in runtime language. Intrinsically typed frames and metaframes allow to define the reduction relation and show how it preserves types. Since both frames and metaframes are typed, we statically know which effects are handled by the frames. We defined reduction rules for this language in such a way that immediately yields preservation. This formalisation could be further expanded to prove the language correctness fully and its translation to other targets.

Bibliography

[1] Guy Lewis Steele, Gerald Jay Sussman. The Revised Report on SCHEME: A Dialect of LISP. In MIT Artificial Intelligence Memo 452, 1978. URL: https://dspace.mit.edu/handle/1721.1/6283.

[2] Olivier Danvy, Andrzej Filinski. Abstracting control. In LISP and Functional Programming, pages 151-160, 1990. URL: https://dl.acm.org/doi/10.1145/91556.91622

[3] Matthias Felleisen. The theory and practice of first-class prompts. In POPL '88: Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages, pages 180-190, 1988. URL: https://dl.acm.org/doi/10.1145/73560.73576

[4] Olivier Danvy, Andrzej Filinski. A functional abstraction of typed contexts. In DIKU Rapport 89/12, DIKU, Computer Science Department, University of Copenhagen, Copenhagen, Denmark. July 1989.

[5] R. Kent Dybvig, Simon Peyton Jones, and Amr Sabry. A Monadic Framework for Delimited Continuations. In Journal of Functional Programming 17, no. 6 (2007): 687–730. 2007. URL: https://doi.org/10.1017/S0956796807006259

[6] Maciej Piróg, Piotr Polesiuk, Filip Sieczkowski. Typed Equivalence of Effect Handlers and Delimited Control. In 4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019), pages 30:1-30:16. 2019. URL: https://doi.org/10.4230/LIPIcs.FSCD.2019.30

[7] Kazuki Ikemori, Youyou Cong, and Hidehiko Masuhara. Typed Equivalence of Labeled Effect Handlers and Labeled Delimited Control Operators. In International Symposium on Principles and Practice of Declarative Programming (PPDP 2023), October 22–23, 2023, Lisboa, Portugal. ACM, New York, NY, USA, 13 pages. 2023. URL: https://doi.org/10.1145/3610612.3610616

[8] Ningning Xie, Youyou Cong, Kazuki Ikemori, and Daan Leijen. First-Class Names for Effect Handlers. Proc. ACM Program. Lang. 6, OOPSLA2, Article 126 (October 2022), 30 pages. 2022. URL: https://doi.org/10.1145/3563289

[9] Patrycja Balik, and Piotr Polesiuk. Unpublished notes. 2024

[10] A.K. Wright, M. Felleisen. A Syntactic Approach to Type Soundness. In Information and Computation, Volume 115, Issue 1, Pages 38-94, 1994. URL: https://doi.org/10.1006/inco.1994.1093

[11] Robert Harper. Practical Foundations for Programming Languages. Cambridge University Press, New York, NY, USA, 2nd edition, 2016

[12] The Agda Development Team. The Agda manual – release 2.8.0. 2025. https://agda.readthedocs.io/en/v2.8.0/

[13] Daggitt, M.L., Allais, G., McKinna, J., Abel, A., Doorn, V., Wood, J., Norell, U., Kidney, D.O., Meshveliani, S., Stucki, S. and Carette, J. The Agda standard library: version 2.0. Journal of Open Source Software, 10(116), p.9241. 2025

[14] Benjamin C. Pierce. Types and programming languages. MIT press, 2002.